

CSC 220 Assignment 02 Stack and Queue Applications

Assigned Tuesday September 29 2026 • Due Monday October 12 2026 at 11:59 pm • 100 points

Work **individually or in a group of two**.

No group may have more than two members.

Build your own generic doubly linked list, use it to implement a stack and a queue, and apply each structure to a small interactive program.

Every group member must be able to explain the complete implementation.

Objective

One goal of this assignment is to practice **guided independent learning**. Use the sources in the **Reading** section to learn how stacks and queues behave. Bring questions to class or office hours, then study Parts 1–3 and make a work plan with your partner, if you have one. Set milestones for implementation and testing so you can complete the assignment by **Monday, October 12 at 11:59 pm**.

Reading

Review stack and queue behavior before coding:

- **Doubly Linked List:** <https://www.codecademy.com/article/doubly-linked-list>
- The Geek Planets, “Stacks and Queues in Java: A Brief Overview of Their Concepts and Implementations” — <https://thegeekplanets.medium.com/stacks-and-queues-in-java-a-brief-overview-of-their-concepts-and-implementations-724ab7b031d0>
- Scaler Topics, “Stack and Queue in Java” — <https://www.scaler.com/topics/java/stack-and-queue-in-java/>

These readings introduce the concepts. Your submission must use **your own linked-list implementation**.

Part 1 Build the data structures

Implement a **generic DoublyLinkedList<T>**. Each node stores a value and references to the next and previous nodes. Keep head, tail, and size accurate after every operation. Build generic **MyStack<T>** and **MyQueue<T>** classes that delegate storage to this list.

Class	Required operations
DoublyLinkedList<T>	addFirst, addLast, removeFirst, removeLast, peekFirst, peekLast, isEmpty, size
MyStack<T>	push, pop, peek, isEmpty, size
MyQueue<T>	enqueue, dequeue, peek, isEmpty, size

- Implement `Node<T>` as a private nested class inside `DoublyLinkedList<T>`. Stack, queue, and application code must use the list's public methods rather than access nodes directly.
- Stack operations must follow **LIFO (Last In First Out)**; queue operations must follow **FIFO (First In First Out)**. You may choose which list end represents the stack top, but document your choice. Queue additions go to the back and removals come from the front.
- **Do not use** `java.util.Stack`, `Queue`, `Deque`, `LinkedList`, or another built-in collection as the backing storage.
- Decide how removal or peek on an empty structure behaves; document and handle it consistently.
- Test the list directly, then demonstrate both stack and queue with `Integer` and `Float` values.

Part 2 Student Task Manager

- Read initial tasks from a text file, one task per line, and enqueue them in file order. This is a FIFO queue; there are no priorities. For example, a file could contain CS Homework 01, Washing Clothes, Attend Seminar, Email a Friend, and Call Mom on separate lines.

```
***** sample input file *****
CS Homework 01
Washing Clothes
Attend Seminar
Email a Friend
Call Mom
*****
```

- After loading, repeatedly show a menu with **Add task**, **Execute next task**, **View next task and pending count**, and **Exit**. Adding a task enqueues it at the back. Executing dequeues the front task and prints the task being completed. Return to the menu after each choice. If the user tries to execute while the queue is empty, print:

"Hurray! You are ahead of yourself and completed all the expected tasks."
- Keep the program open so another task can be added.
- **Test** the initial file order, tasks added during execution, the one-item case, and the empty queue.

Part 3 Library Returns

- The librarian keeps returned book titles in a stack. Initial books may come from a text file (one title per line) or be entered through the menu. Repeatedly offer Submit a book, Shelf the top book, View the top book and waiting count, and Exit.
- Submitting pushes a title; shelving pops and prints the title. The most recently submitted book must be shelved first. If no book is waiting, say that the librarian is caught up and waiting for another return, then keep the menu open.

***** sample input file *****

The Hobbit

Pride and Prejudice

The Martian

A Wrinkle in Time

The Hunger Games

- **Busy-stack rule:** When a push causes the stack size to exceed 15, pause acceptance of new books and automatically pop and print three books from the top. Then resume the regular menu. Apply this rule again if a later push raises the size above 15. Implement the rule in the application, not inside `MyStack<T>`.
- Test LIFO order, empty and one-book cases, new returns between shelving actions, and the busy-stack rule. Show exactly which three books are removed when the rule triggers.

Project organization and submission

- Use appropriate packages. A suggested layout is `datastructures/DoublyLinkedList.java`, `MyStack.java`, and `MyQueue.java`; `applications/StudentTaskManager.java` and `LibraryReturns.java`; plus test code, sample input, and `README.md`. You may choose different names if the structure remains clear.
- Submit one ZIP file containing source files, sample input, tests or labeled test runs, and `README.md`. Exclude compiled `.class` files and `bin/out` directories. In the `README`, provide exact compile and run commands, the input-file format and path, your empty-structure behavior, and each group member's name and contribution. Be prepared for a short individual explanation of a pointer update and a stack or queue trace.

Grading rubric

Testing evidence may be test code or clearly labeled runs showing expected and actual results. We will check both data structure behavior and the application edge cases.

Component	Points
Generic DoublyLinkedList<T> implementation	10
Direct doubly linked list tests	5
MyStack<T> implementation	10
MyQueue<T> implementation	10
Stack and queue checks with Integer and Float	10
Task manager file reading and initial enqueue	5
Task manager menu and behavior	10
Task manager tests	10
Library returns menu and behavior, including busy-stack rule	10
Library returns tests	10
README with compile and run instructions, code structure (relevant package)	10
Total	100